

2025 年中国研究生数学建模竞赛

A 题

通用神经网络处理器下的核内调度

基于驻留段约束的缓存与流水协同优化

历史真题研究阅读版

本稿用于学习研究，不是正式参赛提交版。

正文采用 12 pt 字号与约 18 pt 基线间距，便于阅读与复核。

基于驻留段约束的缓存与流水协同优化

摘 要

针对通用神经网络处理器中拓扑依赖、有限连续缓存与多执行单元并行的耦合，建立以生命周期收缩、驻留段分配和固定驻留图重排为核心的调度方法。在六个题面计算图上，分别求解最小驻留、低额外搬运和搬运预算下的执行时间优化。下述六图结果依次对应卷积小图、大图，注意力小图、大图及矩阵乘小图、大图。

对于问题一，以缓冲区活跃区间建立峰值目标，证明在申请为根、释放为叶的条件下，固定操作顺序的生命期收缩不增加峰值，再比较编号、窗口贪心与深度优先等八种顺序。卷积小图峰值由 7560 降至 6984，其余五图分别为 14048、4884、7972、9728、35328。通过九节点小例的 126 个合法拓扑序验证收缩过程，并用分类型驻留轨迹解释局部贪心与完整峰值的差异。

对于问题二，用四状态描述逻辑生命与物理驻留，结合连续区间最佳适配及未来使用距离选择换出对象；受保护对象形成碎片时，以显式换出和重装恢复空间，并计入全部代价。六图额外搬运量依次为 44766、75546、3620、32852、34944、460800，卷积小图相对编号基准降低 48.90%。同顺序的换出准则对照表明，考虑单次搬运代价并不普遍优于单纯未来距离。

对于问题三，在搬运量零增量约束下比较地址轮换，并在含地址复用依赖的固定驻留图上实施列表重排。证明该重排保留地址互斥及换出集合，最终时间分别为 465892、1044075、37744、150499、173274、1749773 周期，相对问题二降低 1.30% 至 33.38%。注意力大图的向量单元忙碌比例由 46.46% 提高至 69.74%。进一步区分原始工作下界与固定驻留下界，六图相对后者的差距为 1.53% 至 24.95%，仅用于评价保留当前驻留关系后的改进空间。

所有方案经原始依赖、连续地址、使用时驻留和执行单元互斥重算。分类型搬运、重复换出及局部流水对照表明，缓存压力集中位置与地址链结构共同影响三问的优化收益。结果适用于六个题面计算图和本文采用的驻留段复用模型。

关键词：有向无环图；生命周期；连续地址分配；换入换出；流水调度

1. 问题重述

题目将算子细化为由原子指令构成的有向无环图，节点包括指定执行单元上的操作，以及缓冲区申请和释放操作 [1]。每个缓冲区只属于一种缓存，需占据长度确定的连续地址区间。不同执行单元可以并行工作，同一执行单元只能串行执行。数据依赖和物理地址复用都会引入等待，因此拓扑顺序、缓存位置与实际完成时间必须联合考虑。

问题一要求在不考虑具体缓存容量限制时，寻找最大累计驻留量尽可能小的拓扑序。问题二在给定各类缓存容量后，为缓冲区确定偏移，必要时加入换出及换入操作，使额外搬运量尽可能小。问题三则允许进一步调整顺序和分配，在搬运量不显著增加的前提下降低执行时间。三个问题分别突出存储压力、物理可行性和执行并行性，目标之间存在竞争。

本文对六个输入图使用相同算法组合，不以算子名称编写专用调度规则。题面未给出“不显著增加”的数值阈值，因此问题三采用更严格的搬运量零增加约束。研究重点是获得可核验的完整调度方案，并明确启发式与评价模型的适用边界。

2. 数据理解与总体分析

2.1 输入结构与审计

输入同时提供节点和边的 JSON、CSV 表示。节点编号用于识别对象，不代表最优执行次序。操作节点的缓冲区列表同时包含输入与输出，不能仅依据列表顺序推断读写属性。本文直接使用题面给出的依赖边约束生产与消费先后。

六图规模如下。读取时逐字段比对两种表示，检查节点编号、重复边、缓存类型、大小及申请释放配对。全部输入通过检查，没有对缺失值进行填补，也没有删除任何节点或依赖。

表 1: 输入计算图规模

计算图	节点数	依赖边数
Conv_Case0	2580	3869
Conv_Case1	36086	85653
FlashAttention_Case0	1716	2712
FlashAttention_Case1	6952	11184
Matmul_Case0	4160	7104
Matmul_Case1	30976	55040

2.2 操作规模与缓冲区共享

节点总数包含申请与释放。每个缓冲区恰有一对管理节点，因此操作数满足 $|V_o| = N - 2|B|$ 。表 2 把真正占用执行单元的操作与缓冲区分别计数，其中“多使用者对象”表示被至少两个操作引用的缓冲区，使用者同时包括生产与消费操作。该指标描述引用范围；判断数据复用还需结合操作属性和依赖关系。

表 2: 操作规模与缓冲区引用结构

计算图	操作数	缓冲区数	多使用者对象	最多使用者
卷积小图	918	831	823	16
卷积大图	12060	12013	12013	97
注意力小图	572	572	540	5
注意力大图	2296	2328	2200	9
矩阵乘小图	1728	1216	1216	9
矩阵乘大图	13056	8960	8960	17

卷积大图有 12060 个操作、12013 个缓冲区，单个缓冲区最多被 97 个操作引用。固定某一操作之后，共享对象仍可能服务于多个后继分支，其释放时刻取决于这些分支中最后完成的使用者。因而用“本操作完成”作为释放依据会丢失后续数据；求解时需要维护每个对象的剩余使用数，并同时检查释放节点的显式前驱。卷积小图的最大使用者数为 16，与大图存在明显差别，后文分别考察两图的驻留组成和重复换出。

矩阵乘小图和大图分别包含 512、4096 个矩阵乘操作，级间移动操作分别为 1024、8192 个。矩阵计算前的两个输入通常由级间搬运提供，因而单纯提前矩阵操作的排序优先级，仍会受到数据到达时刻的约束。两图的缓冲区分别有 1216、8960 个，其中一级缓存对象为 128、512 个，零级左右输入缓存对象合计为 1024、8192 个。对象数量分布与空间压力分布需要分别测量：大量短期零级对象可以分时复用地址，较少而寿命较长的一级对象也可能形成主要驻留峰值。

注意力小图和大图分别包含 64、256 个矩阵乘操作，同时具有行最大值、指数、行求和、乘法与加法等向量操作。其统一缓存对象分别为 356、1512 个，占全部对象的较大部分。这一结构使矩阵单元与向量单元之间的数据交接成为流水分析的重要对象。两个阶段可以由不同硬件单元承担，但后一阶段仍须等待所需数据及缓存地址同时就绪。后文选择注意力小图展示局部时间窗，是因为同一窗口能够观察矩阵计算、级间搬运和向量计算的先后关系。

六图具有不同的操作组成，求解过程仍统一读取节点的执行单元、周期和缓冲区属性。操作名称用于识别管理节点及题面规定的输入复制计费，其余排序与分配由图结构和资源状态确定。这样可以在同一组规则下比较不同计算图，避免将某一图的结果建立在专门编写的顺序上。

2.3 目标的耦合关系

缓存驻留峰值低，并不意味着每种缓存都能分配成功。一方面，不同类型不能互相借用空间；另一方面，分散的空闲区间无法容纳较大的连续申请。与此相反，过度追求小峰值可能使原本可重叠的计算串行化。因此首先通过生命周期收缩建立基准，再分别围绕搬运量和时间评价候选。

采用“拓扑顺序—连续分配—依赖重建—流水评价”的计算链。问题一输出原始节点的置换及其峰值，问题二在候选顺序上增加地址和换入换出事件，问题三利用完整方案中的结构依赖重排各单元指令。三问的接口依次增加物理约束，使每一步产生的中间对象都具有明确含义。

同一操作顺序可以对应不同的连续地址方案。即使两种地址方案引起的搬运量相同，地址复用边也可能不同，进而改变跨单元等待。反过来，减小峰值的顺序可能推迟本可提前加载的数据。本文分别计算峰值 H 、额外搬运量 D 和完成时间 T ，在问题二按搬运量优先选择，在问题三以该搬运量为上限比较时间。这样的分层评价保留了三问的目标差别，也使无改进的候选能够用于分析目标之间的冲突。

实验以六个原始计算图为共同输入，所有算法采用相同容量、搬运计费与时间递推。方案生成后，从完整调度文本重新构建驻留和流水，核对节点唯一性、原始边、地址互斥及执行时间。验证对象是可执行的完整方案；汇总表据这些方案计算得到。参数比较保持同一输入与评价方式，具体控制条件在第八节给出。

3. 模型假设与歧义处理

缓存大小沿用题面的抽象容量单位，不将其转换成字节；执行时间统一使用时钟周期。缓存申请与释放耗时为零，操作耗时固定为节点的周期字段。原始依赖边均不可删除，也不依据真实芯片的额外机制添加题面没有给出的吞吐假设。

问题一的驻留量按所有缓存类型合计计算，而非各类型峰值的最大值。对同一类型同时驻留的缓冲区要求地址不重叠，不同类型拥有独立地址空间。

附录 C 通过前一缓冲区的释放节点约束后一缓冲区的申请节点，直接描述了未发生换出的地址复用。发生换出时，物理地址在换出结束后即可复用，缓冲区的逻辑生命则持续到原始释放节点。本文将两者区分为逻辑生命周期与物理驻留段：申请或换入开启一段驻留，释放或换出结束该段。新驻留段必须等待所复用旧段的结束。若仍以最终释放节点约束换出期间的所有复用，可能人为形成依赖环，无法表达题面允许的换出机制。因此驻留段是本文对该情形的明确补充解释；与未提供的官方评测程序的一致性尚不能确证。

换出目标曾被任一输入复制节点引用时，按题面规定只计换入搬运量，换出时间为零；不自行判断数据是否被后续修改并改变计费方式。所有缓存类型均依题面统一换出规则处理。问题三要求额外搬运量不高于问题二选定方案，从而避免人为设定宽松的容忍阈值。

4. 符号说明

容量与搬运量均使用题面容量单位；时间及工作量使用时钟周期。节点编号、序列位置和地址偏移为整数。缓存类型集合记为 $\mathcal{T}$ 。本文用 E_e 表示结束时间，以免与边集 E 混淆。

表 3: 主要符号及取值

符号	定义与取值
$G = (V, E), V_o$	原始有向无环图、非管理操作节点集合
N, B, B_v	原始节点数、缓冲区集合、操作 v 引用的去重缓冲区集合
s_b, t_b, C_t	正整数缓冲区大小、类型及该类型容量
a_b, f_b	缓冲区 b 唯一的原始申请、释放节点
$\pi(v)$	节点位置；问题一为 V 到 $\{1, \dots, N\}$ 的双射
x_{bj}, o_{bj}, z_{bj}	第 j 个驻留段的非负整数偏移、开始与结束节点
χ_b	被任一 COPY_IN 引用时取 1，否则取 0
$\mathcal{M}, M$	换出事件集合及事件数；同一缓冲区可重复出现
H, D, T	最大逻辑驻留量、额外搬运量、总执行周期
c_v, p_v	非负执行周期、指定执行单元；管理节点无独占单元
S_v, F_v, r_v	非负开始时间、结束时间、下游最长路径长度
L_H, L_0, L_R	同时使用容量下界、原始工作下界、固定驻留图时间下界
W, α	就绪窗口大小、启发式评分权重

5. 最小缓存驻留调度

5.1 生命周期模型与下界

一个缓冲区在申请后持续占用逻辑缓存，直到全部使用结束并执行释放。调度的作用是改变这些区间的相互重叠：过早申请会延长区间，过晚释放会保留已经无用的数据，而不同计算分支的交错还会改变末次使用位置。因此本问先处理管理节点的可移动性，再搜索操作节点的相对顺序。

问题一只优化节点顺序，不给缓冲区设置物理偏移，也不加入换入换出。缓冲区从申请后到释放前始终计入驻留量。对一个节点置换 π ，定义处理完序列第 k 个节点后的活跃指示量

$$y_b(k; \pi) = \mathbf{1}\{\pi(a_b) \leq k < \pi(f_b)\}, \quad k = 0, \dots, N. \quad (1)$$

于是调度模型为

$$\min_{\pi} \quad H(\pi) = \max_{0 \leq k \leq N} \sum_{b \in B} s_b y_b(k; \pi), \quad (2)$$

$$\text{s.t.} \quad \pi(u) < \pi(v), \quad (u, v) \in E, \quad (3)$$

$$\pi(a_b) < \pi(v) < \pi(f_b), \quad v \in V_o, b \in B_v, \quad (4)$$

$$\pi : V \longrightarrow \{1, \dots, N\} \text{ 为双射.}$$

式 (4) 把使用时必须活跃的语义显式列出；在输入依赖充分描述生命期时，它由原始路径蕴含。申请记正增量、释放记负增量的题面定义，与式 (2) 完全一致。总峰值是在同一位置对各类型求和后取最大值，不能相加各类型在不同位置达到的峰值。

每条操作的全部输入和输出缓冲区必须同时存在。因此，无论如何重排，都有

$$H^* \geq L_H := \max_{v \in V_o} \sum_{b \in B_v} s_b. \quad (5)$$

此界利用单个操作的同时使用需求。对任一可行序列，在操作 v 发生前，其缓冲区均已申请且尚未释放，因此该时刻的驻留量至少为右端求和；再对全部操作取最大值得到式 (5)。当可行候选达到 L_H 时，上下界闭合，峰值最优性随之成立。多个操作之间必须继续保留的对象会使实际下界更高，故后文同时给出可行上界及这一可直接计算的下界。

也可以从执行前沿理解峰值。将某一操作前缀记为 P ，其中已经开始使用、仍被后续操作需要的缓冲区构成跨前沿集合。它们把已经完成的计算与尚未完成的计算连接起来，生命期收缩后仍须保留。选择下一个就绪操作，会同时改变首次使用对象和末次使用对象；局部评分衡量这一即时变化，完整峰值则取决于所有前沿中最大的驻留总量。

设当前驻留为 R ，本步全部新增申请总长为 $\tilde{n}_v$ ，操作后实际释放总长为 d_v^{real} 。全部新增申请包括使用对象和额外显式申请前驱。该步驻留先增至 $R + \tilde{n}_v$ ，随后降至 $R + \tilde{n}_v - d_v^{\text{real}}$ ，历史峰值更新为

$$H_{\text{new}} = \max\{H_{\text{old}}, R + \tilde{n}_v\}. \quad (6)$$

释放发生在操作之后。即使 $\tilde{n}_v - d_v^{\text{real}}$ 很小，本步也可能先经历较大的申请峰值。这一区别是局部净增量与全局峰值不能直接等同的原因，也是保留完整重放评价的依据。

5.2 固定操作顺序下的生命周期收缩

题面规定申请节点为根、释放节点为叶，六图均满足这一结构。固定非管理操作的相对顺序后，一个申请必须早于其所有显式后继以及所有引用该缓冲区的操作。称其中最早出现的节点为首次需求节点。相应地，释放必须晚于全部显式前驱和全部使用者，称其中最晚出现的节点为末次需求节点。这里不能仅以第一次读或最后一次读代替需求节点，否则可能遗漏额外管理依赖。

收缩性质。在上述前提下，将申请移动到首次需求节点之前，将释放移动到末次需求节点之后，不会增加驻留峰值。

证明分两步。首先，申请是根节点，向后移动不会违反其入边；又因为尚未跨过任何显式后继或使用者，其出边和使用约束仍成立。将它与中间节点逐个交换，只会删除被跨越区间内这一缓冲区的正容量贡献。其次，释放是叶节点，向前移动不会违反其出边；停止在全部前驱和使用者之后，则其入边不受影响。逐个交换同样只会减少中间区间的驻留量。其他缓冲区的申请、释放相对关系不变，因此每一步交换的最大累计量均不增加。对所有缓冲区重复此变换，得到同一操作顺序下不劣的规范化序列。

收缩后，每个对象的区间端点贴近需求范围，搜索因而集中到操作相对顺序。共享对象的释放仍以末次使用和全部显式前驱完成为条件。对于申请有入边或释放有出边的扩展模型，可移动端点还需受到这些边的限制；本文六图满足根申请、叶释放条件。

5.3 九节点算例：从延迟申请到峰值闭合

构造三个操作 $u \rightarrow v \rightarrow w$ ，缓冲区 a, b, c 的大小分别为 4、3、5。操作 u 使用 a ， v 使用 a, b ， w 使用 b, c 。每个对象分别设申请节点 A_a, A_b, A_c 和释放节点 F_a, F_b, F_c ；申请指向全部使用者，使用者指向对应释放。此例共九个节点，用于检验生命周期模型，不计入六个题面图的实验统计。

若先申请全部对象，再依次执行三个操作，则申请阶段驻留依次为 4、7、12，峰值已经达到 12。即使此后每次操作完成就尽早释放，该峰值也无法撤销。将同一操作顺序收缩后得到

$$A_a, u, A_b, v, F_a, A_c, w, F_b, F_c. \quad (7)$$

表 4 列出每个节点后的状态，使申请与释放的贡献可以逐步复算。

表 4: 构造小例的收缩序列与驻留量

A_a	u	A_b	v	F_a	A_c	w	F_b	F_c
4	4	7	7	3	8	8	5	0

操作 w 同时需要大小为 3 和 5 的两个对象，所以 $L_H = 8$ 。表中峰值恰为 8，因而这一构造例的最优值为 8。对其全部合法拓扑序进行递归枚举，共得到 126 个序列，其中 120 个峰值为 12，6 个峰值为 8，与下界及手算一致。枚举过程中每次只从全部前驱已输出的节点中选择，逐节点更新驻留和峰值，输出终止时检查驻留归零。

这一小例把两个层次区分开：固定操作链时，延迟申请便能排除不必要的三对象重叠；大图还存在多个可选操作分支，需要进一步搜索操作顺序。枚举方法适合核验小例，随着可交换节点增加，候选数迅速增长，六图因此采用受限候选生成。小例中的最优性由上下界与穷举共同支持，大图的解质量则由实际基准和对应下界评价。

5.4 候选生成与算法对应

在操作诱导子图中维护入度和就绪集合。每选择一个操作，先插入尚未出现且为该操作所需的申请，随后发出操作，更新剩余使用次数；只有剩余使用次数为零且全部显式前驱已完成时才插入释放。输出完毕后逐边检查式 (3)，并检查节点置换及累计量首尾归零。

为了分开不同调度偏好的效果，设置三类主要操作顺序。编号顺序每次选择最小编号的就绪节点，是经过生命期收缩的强基准，而非未经处理的文件行顺序。缓存贪心先选就绪集合中编号最小的 W 个节点，再最小化

$$q_v = n_v - \alpha d_v, \quad n_v = \sum_{b \in B_v \setminus B_{\text{live}}} s_b, \quad d_v = \sum_{b \in B_v: \text{remain}(b)=1} s_b. \quad (8)$$

其中 d_v 是末次使用带来的释放潜力；若还有额外释放前驱未完成，实际释放仍须等待，评分不强制兑现这一潜力。相同评分按新增量、下游路径及编号打破平局。比较 $(W, \alpha) = (32, 1), (128, 1), (64, 2)$ ，窗口并非从所有就绪节点中寻找评分最好的 W 项。

第三类从编号有序的终端操作出发，沿前驱作深度优先访问，以完成访问顺序得到拓扑序；前驱正序和逆序各形成一个候选。它有机会缩短单个依赖分支的持续时间，但共享缓冲区可能使分支优先与复用优先发生冲突。另保留两种流水评分产生的操作顺序参与峰值比较。最终八种顺序均经过相同生命期收缩，再按真实峰值选优，不用局部评分直接替代目标函数。

令 $Q = \sum_{v \in V_o} |B_v|$, $K = \max_v |B_v|$ 。图遍历及未排序的深度优先核心为 $O(N + |E|)$ ；实现中的编号排序另有至多 $O((N + |E|) \log N)$ 开销。窗口贪心每步扫描就绪集合，并计算至多 W 个缓冲区评分，保守界为 $O(N^2 \log(W + 1) + NWK + |E| + Q)$ 。空间为 $O(N + |E| + Q)$ 。实际并未枚举所有拓扑序，因此计算可承受性以限制搜索范围为代价。

5.5 状态更新与停止条件

算法的输入是图、窗口 W 和释放权重 α ，输出是包含全部原节点的序列。状态由已输出节点集合、操作剩余入度、就绪集合、逻辑活跃对象和剩余使用次数组成。操作剩余入度只统计操作间的边；管理依赖在输出申请和释放时单独检查。逆拓扑遍历预先计算下游周期长度，用于评分相同时优先处理后续链条较长的操作。

每次迭代先从就绪集合构造候选窗口，按所选评分确定操作 v 。随后插入 v 尚缺的缓冲区申请，以及指向 v 而尚未输出的其他申请前驱。全部前驱满足后输出 v ，将其引用对象的剩余使用次数各减一，并检查因此就绪的释放。最后更新操作后继的入度，将降至零的后继加入就绪集合。表 5 列出这些步骤各自维持的条件。

表 5: 操作顺序生成中的状态更新

步骤	更新内容	维持条件
选择操作	按窗口与评分选取就绪节点	全部操作前驱已经输出
补齐申请	输出未申请的需求对象	操作发生前对象均活跃
执行与计数	输出操作，递减使用计数	计数等于未输出使用者数量
检查释放	计数归零且显式前驱完成才释放	后续操作仍需对象保持活跃
扩展前沿	后继入度归零后进入就绪集	每个操作恰好输出一次

逻辑活跃集合与剩余使用计数承担不同职责。首次使用对象需要插入申请，已经分配但未来仍要使用的对象持续保留；计数归零时还要检查显式释放前驱。这样即使图中存在额外管理边，释放也不会越过约束。遍历到就绪集合为空时，处理剩余的纯管理节点，并以全节点唯一、全部边正序及最终驻留为零作为完成条件。

例如，当前驻留量为 10，两个就绪操作的 (n_v, d_v) 分别为 $(4, 6)$ 和 $(1, 2)$ 。当 $\alpha = 1$ 时，两者净评分为 -2 和 -1 ，缓存贪心优先前者；按式 (6)，本步申请峰值却分别为 14 和 11。前者操作后的剩余驻留更小，后者瞬时申请压力更小，局部准则在这两个倾向之间作出取舍。后续哪个分支会带来更多驻留，还取决于尚未执行的使用者，因此最终必须以完整序列的最大值评价。

5.6 六图峰值与解质量

表 6 同时给出收缩后编号基准、选定峰值及式 (5) 的下界。比值 H_1/L_H 是相对一个可计算下界的比值，不是已知的最优性误差。

表 6: 驻留峰值与同时使用下界（题面容量单位）

计算图	H_{id}	H_1	L_H	H_1/L_H
卷积小图	7560	6984	3144	2.22
卷积大图	14048	14048	932	15.07
注意力小图	4884	4884	512	9.54
注意力大图	7972	7972	512	15.57
矩阵乘小图	9728	9728	512	19.00
矩阵乘大图	35328	35328	512	69.00

卷积小图通过深度优先顺序从 7560 降至 6984，减少 576，降幅为 7.62%。其余五图在本轮候选中保留编号基准，选定峰值分别为 14048、4884、7972、9728 和 35328。矩阵乘大图的同时使用下界为 512，与 35328 的可行峰值之间仍有较大距离。该界主要刻画单次操作，跨操作保留对象则由后续驻留组成进一步说明。

5.7 驻留轨迹与峰值形成位置

为观察顺序改变后的过程，图 1 给出卷积小图三种候选的完整驻留轨迹，并放大编号与深度优先方案的峰值附近。横轴为已处理的原始节点位置，纵轴为所有类型容量之和。同一横坐标表示相同的已处理节点数量，两种序列在该位置的具体节点可以不同；因此比较首先围绕各自的峰值及组成展开。

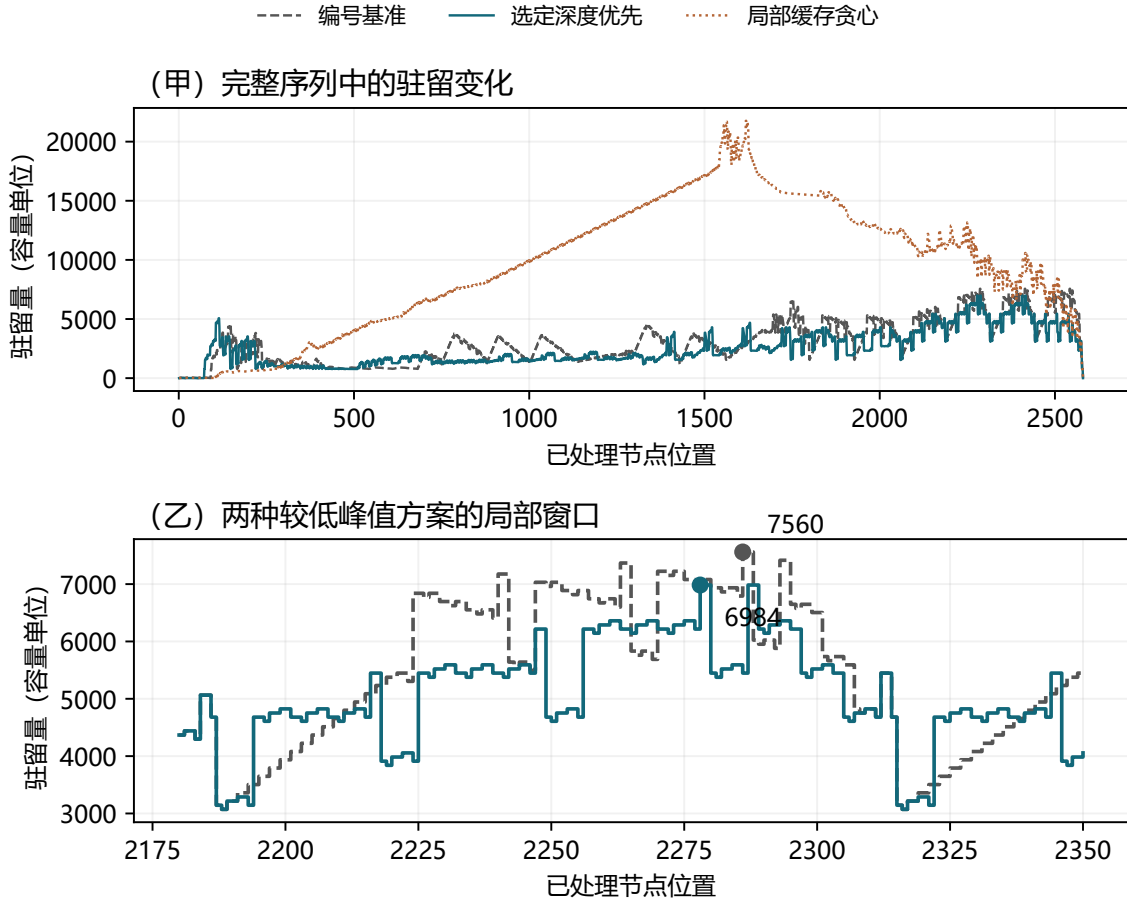


图 1: 卷积小图的驻留轨迹与峰值窗口

编号基准在第 2286 个节点之后达到 7560，此时节点为 1485，活跃对象共 13 个，其中一级缓存占 7488，零级右输入缓存占 72。深度优先方案在第 2278 个节点之后达到 6984，此时节点为 1483，活跃对象共 9 个，两个类型的容量分别为 6912 和 72。两方案峰值处的零级贡献相同，总峰值的 576 差值与一级缓存贡献的差值一致。这一分解把总体收益定位到峰值窗口的类型组成，进一步逐对象追踪才可判断每个对象的申请或释放变化。

局部缓存贪心在 $(W, \alpha) = (32, 1)$ 时达到 21762 的峰值，出现在第 1618 个节点之后。该处活跃对象增至 283 个，一级缓存占 5378，而零级右输入缓存占 16384。由此可见，较大的峰值并非仅将原基准的一级缓存高峰继续抬升，而是伴随另一缓存类型的大量逻辑对象同时存续。问题一尚未进行物理分配，16384 表示该顺序要求的逻辑驻留量；后续引入物理容量时，必须通过换入换出处理这种压力。

这与净增量评分的结构相吻合。评分针对当前就绪操作计算首次需求与释放潜力，未显式惩罚某一类型已积累的大量活跃对象。连续选择单步代价较低的操作，可能保留许多还需其他分支使用的中间缓冲区。图中的高峰和对象数量为这一风险提供了具体过程证据，因而候选筛选采用真实峰值，而非累加局部评分。

表 7: 各选定序列总峰值发生位置的分类型组成

计算图	$L1$	UB	$L0A$	$L0B$	$L0C$
卷积小图	6912	0	0	72	0
卷积大图	14040	0	0	8	0
注意力小图	3072	1044	128	128	512
注意力大图	5120	2084	128	128	512
矩阵乘小图	9216	0	128	128	256
矩阵乘大图	34816	0	128	128	256

表 7 中的每一行取自该图总峰值的同一节点位置，行内相加恰为 H_1 。卷积大图的 14048 中有 14040 来自一级缓存；矩阵乘两图的一级缓存贡献分别为 9216、34816。注意力两图则同时出现一级缓存和统一缓存的较高占用，小图在总峰值位置的统一缓存需求为 1044，大图为 2084，均高于后续问题二的 1024 容量。这些结果说明，分类型压力对连续分配具有直接意义，单个总峰值不足以决定哪类对象需要换出。

问题一由此给出六份可行原始序列、对应峰值及可计算下界，并揭示编号基准、分支顺序和局部贪心的差异。进入问题二时，保留不同顺序候选用于真实地址分配，使其价值由额外搬运量重新评价。这样既利用了生命期收缩，也保留了不同目标下的选择余地。

6. 连续缓存分配与换入换出

6.1 逻辑生命与物理驻留状态

上一问把缓冲区生命期表示为逻辑区间。本问为这些对象分配真实连续地址，一个逻辑生命期因换出可以拆成若干段物理驻留。每次换出释放片内区间，后续使用前再恢复对象，恢复地址可以改变。因此求解同时维护“数据还需保留”和“数据当前占据哪一片地址”两种信息。

给定规范化顺序后，问题二还需要确定初始偏移、换出对象及其插入位置、配对换入位置和新偏移。问题允许进一步调整原始操作顺序，因此这些选择共同构成可行域。缓存容量分别为 $C_{L1} = 4096$ 、 $C_{UB} = 1024$ 、 $C_{L0A} = C_{L0B} = 256$ 、 $C_{L0C} = 512$ ，类型之间不能借用空间。

每个缓冲区在任一序列位置处于四种状态之一：未申请、驻留、已换出、已释放。合法状态迁移为

$$\text{未申请} \xrightarrow{\text{ALLOC}} \text{驻留} \begin{matrix} \xrightarrow{\text{SPILL_OUT}} \text{已换出,} \\ \xleftarrow{\text{SPILL_IN}} \end{matrix} \text{驻留} \xrightarrow{\text{FREE}} \text{已释放.} \quad (9)$$

只有驻留状态允许操作引用；已释放后不可再次申请同一标识。换出不终结逻辑生命，仍须保留原始 FREE。若换出后已无普通操作需要该对象，也必须完成配对换入后再释放，以符合新增节点和配对规则，而不能删去换入以降低指标。

用序列区间 $[\pi(o_{bj}), \pi(z_{bj}))$ 表示第 j 段物理驻留，从 ALLOC 或 SPILL_IN 开始，到 FREE 或 SPILL_OUT 结束。其地址为整数半开区间 $[x_{bj}, x_{bj} + s_b)$ ，满足

$$x_{bj} \in \mathbb{Z}_{\geq 0}, \quad x_{bj} + s_b \leq C_{t_b}. \quad (10)$$

若两个同类型驻留段在序列上重叠，其地址必须满足析取约束

$$x_{bj} + s_b \leq x_{ak} \quad \text{或} \quad x_{ak} + s_a \leq x_{bj}. \quad (11)$$

每个使用者必须落在相应驻留段内，段的关闭节点等待该段全部使用者，下一次换入等待对应换出。时间上的地址互斥还由下一节的释放到再占用依赖保证；仅在输出序列上画不重叠区间，并不足以证明并行执行时安全。

6.2 搬运目标与连续空间的困难

令 χ_b 按题面 COPY_IN 引用关系取值。问题二的目标是

$$\min D = \sum_{(b,j) \in \mathcal{M}} (2 - \chi_b) s_b, \quad (12)$$

约束为原始节点拓扑顺序、式 (9)–(11) 及换出换入配对。重复换出同一对象按不同事件重复计费。一次换入耗时 $2s_b + 150$ ；换出在 $\chi_b = 1$ 时耗时零，否则也是 $2s_b + 150$ 。这些耗时进入问题三的执行单元工作量，零耗时换出仍是依赖节点。

容量和连续性是两类约束。以容量 10 为例，若两个当前需保留的对象位于 $[0, 2)$ 、 $[6, 8)$ ，剩余空闲区间长 4 和 2；新对象长 5，三者总长仅为 9，却没有一个空闲段可容纳它。因而“空闲总量足够”不是分配成功的充分条件。动态存储分配文献将空闲区间组织与外部碎片作为核心问题 [2]，本文据此同时检查空闲总量与可用连续区间。

记某类型当前最大连续空闲段长度为 $L_t^{\max}$ ，空闲总量为 F_t 。请求大小为 s 时，固定现有偏移下的可分配条件是 $L_t^{\max} \geq s$ ；若 $F_t \geq s > L_t^{\max}$ ，阻塞来自外部碎片；若 $F_t < s$ ，则空闲总量已经不足。两种情形都可能通过换出释放空间，但牺牲对象的位置决定释放后能否合并成足够长的区间。后文的距离评分按对象选牺牲者，连续区间扫描负责确认每次牺牲是否已经达到分配条件。

最佳适配枚举所有长度不小于 s 的空闲段，选择长度最小的一段，并从该段起始位置分配；长度相同再取较低地址。其直接效果是使本次分配在所选空闲段内留下的余量较小，较长空闲段可以暂时保留给后续较大对象。未来申请和释放可能改变区间结构，因此每次仍重新计算空闲段。由此，地址选择是基于当前状态的启发式，与完整调度顺序共同决定最终搬运量。

6.3 分配、换出与恢复算法

先沿原始顺序预存每个对象的未来使用位置队列，再逐节点模拟四状态迁移。遇到申请或换入，扫描对应缓存，优先选择长度最小且足够的空闲区间。若没有可用区间，排除当前操作必需的对象，从其余同类型驻留对象中选择牺牲者，插入换出并合并空闲区间，重复直至分配成功。

牺牲评分比较两种选择：最大化下一次使用距离 Δ_b ，或最大化

$$q_b^{\text{spill}} = \frac{\Delta_b}{(2 - \chi_b)s_b}. \quad (13)$$

无后续普通操作使用的对象，将其距离置为原始序列长度加一。后一规则倾向于换出未来较晚使用且搬运较便宜的对象，连续空闲段的组合效果则在每次换出后重新扫描确认。Bélády 对替换问题的研究利用完整未来引用序列构造离线比较基准 [3]。本文借鉴未来引用信息，在对象大小不等、多对象同时使用及连续地址约束下构造上述两种牺牲评分，并以六图真实搬运量比较其表现。

普通换出可能被受保护对象的碎片阻塞，此时启用显式恢复：换出本次操作所需缓存类型中的全部驻留对象，再按对象大小递减顺序重装本次必需集合。该步骤允许改变偏移，但每次搬运均计入式 (12)，不是零成本地址搬移。当前实现以容量单元数组表示占用情况，便于检查边界；这也是容量不大时选择简单实现的原因。

恢复成功的前提为每个操作在每种缓存上的必需总量满足

$$\sum_{b \in B_v: t_b=t} s_b \leq C_t, \quad v \in V_o, t \in \mathcal{T}. \quad (14)$$

这一前提已对六图逐操作检查。证明如下：将相关类型清空后，在同一类型中把必需对象紧邻排列，其最后一个地址不超过总长，而总长不超过容量，所以存在连续放置。未换入的其他对象可留在 DDR 等待后续使用；DDR 容量在题面中不作限制。在固定原始顺序下，每个节点均可通过有限次换出和重装推进，因而不会因外部碎片永久停滞。若式 (14) 失败，单纯换出策略无法解决同时使用需求，应直接报告不可行，而不是无限循环。

初始偏移对应原始 ALLOC，每次换出另记录新的换入偏移。若有 M 次换出，新增节点恰为 $2M$ 个，第 j 次从零编号时使用 $N + 2j, N + 2j + 1$ ，并保持原始节点各出现一次。未来使用队列的预处理为 $O(N + Q)$ 。计入容量扫描、牺牲对象扫描及恢复时的对象排序，分配阶段的保守时间界为 $O((N + M)(C + |B| + K \log(K + 1)) + Q)$ ，其中 $C = \max_t C_t$ ；显式恢复增加的工作已包含在实际 M 内，不将其当作常数开销。

6.4 一次分配的决策计算与恢复示例

未来使用信息来自已经给定的原始节点顺序。设当前处理位置为 i ，对象 b 的未来使用队列首项为 j_b ，则 $\Delta_b = j_b - i$ 。处理完当前操作后，弹出本次使用位置；以后即使对象暂时换出，其未来队列仍继续保留。这样，逻辑需求与物理状态相互独立，换入时能够找到下一使用需求，而无需重新扫描整个后缀。

两种牺牲规则的差别可以用一个构造状态说明。设同类型两个可换出对象 a, b 的大小分别为 4、2，下一使用距离分别为 12、8； a 为题面输入复制对象， b 为中间对象。两者一次事件的搬运代价均为 4，距离评分和代价评分都优先换出 a 。若把 a 改为中间对象，其代价变为 8，代价评分变为 $12/8 = 1.5$ ，而 b 的评分为 $8/4 = 2$ ，此时距离规则仍选择 a ，代价规则选择 b 。这一计算说明，分母改变的是在未来距离与本次搬运之间的取舍。

牺牲规则还受到保护集合与地址形状的共同限制。若当前操作同时使用 a ，它就从普通牺牲集合中排除，即使其评分最大也不会被普通换出选中。若换出 b 后仍无足够长区间，

算法会继续选择其他合法牺牲者；一次申请因此可能对应多次换出。评分相同时，当前实现优先较大对象，再以较小对象编号打破平局，使相同输入生成相同结果。

在容量 10 的碎片小例中，当前必需的两个对象各长 2，分别位于 $[0, 2)$ 、 $[6, 8)$ ，还需装入一个长 5 的对象。必需总量为 9，满足式 (14)，但原地址布局阻塞请求。若两对象均受保护且没有其他可牺牲对象，普通换出无法推进。显式恢复先将二者换出，再把三个必需对象按长度排列为 $[0, 5)$ 、 $[5, 7)$ 、 $[7, 9)$ ，余下 $[9, 10)$ 空闲。

若被换出的两个对象均为中间对象，该恢复至少增加 $2 \times 2 + 2 \times 2 = 8$ 的搬运量；每个对象的换出、换入各耗时 $2 \times 2 + 150 = 154$ 周期。这些事件分别进入输出和输入搬运单元，实际增加的完成时间还由依赖与重叠决定。该构造例展示了空间可恢复性及其代价，既解释为什么总量可行仍会遇到局部失败，也说明恢复过程需要显式记录搬运。

对每个普通操作，算法先检查各类型必需总量，然后按大小递减确保全部需求对象驻留。若部分对象已经装入而后续请求遭遇保护碎片，此前已发生的换出、换入仍保留，恢复从当前状态继续执行。该处理使搬运统计与实际生成的操作一致。输出 FREE 时若对象尚在 DDR，则先补配对换入，再释放其逻辑生命；因此每次换出最终都有对应恢复，所有驻留状态在遍历结束时归零。

恢复的有限性可由两阶段说明。普通换出每次减少一个当前驻留且未受保护对象，有限对象集合使该循环有限。清空相关类型后，大小递减装入的任一前缀总长不超过完整必需集合总长，故各次申请都可放在剩余连续后缀中。处理完本操作再进入下一节点，整个原始序列有限，从而输出完整有限方案。

6.5 搬运量结果及策略取舍

问题二在编号、三组缓存贪心及两种深度优先顺序上，各比较两种换出评分，共十二个候选。按 D 最小选择，平局取较小执行时间。表 8 的编号基准采用相同生命周期收缩、最佳适配和距离评分，使比较反映顺序与牺牲规则的组合变化，而不是额外预处理差异。

表 8: 问题二的额外搬运量与换出次数

计算图	D_{id}	D_2	降幅/%	换出次数
卷积小图	87612	44766	48.90	56
卷积大图	76636	75546	1.42	2116
注意力小图	3620	3620	0.00	23
注意力大图	32852	32852	0.00	212
矩阵乘小图	34944	34944	0.00	273
矩阵乘大图	460800	460800	0.00	3600

卷积小图的搬运量从 87612 降至 44766，降幅 48.90%；顺序改为深度优先、仍用距离评分时已经降至 45336，再换用代价评分降至 44766。因此主要收益来自该图上的顺序变化，而代价评分进一步节省 570。卷积大图则选择深度优先与距离评分，搬运量为 75546；若对

同一深度优先顺序改用代价评分，搬运量反而升至 81538。这一对照直接说明式 (13) 不是普遍优于距离评分的准则。

两个注意力图和两个矩阵乘图均保留编号基准的搬运量，其中矩阵乘大图有 3600 次换出、搬运量为 460800。换出次数与单次大小共同决定总量，因此下面分别考察事件分布和重复对象，以识别这一开销的具体组成。

6.6 分类型搬运与重复换出

对每个类型 t ，将该类型所有换出事件的式 (12) 贡献求和得到 D_t ，满足 $D = \sum_t D_t$ 。再以对象标识合并事件，记发生过换出的不同对象数为 B_s ，重复事件数定义为 $M - B_s$ 。这两个统计量分别反映空间类型和同一对象被反复移出的程度。表 9 给出问题二方案的实际结果，其他零级缓存类型的额外搬运量在六图中均为零。

表 9: 额外搬运的缓存类型与重复事件

计算图	D_{L1}	D_{UB}	不同对象	重复事件	最大次数
卷积小图	44766	0	28	28	5
卷积大图	75546	0	690	1426	9
注意力小图	0	3620	23	0	1
注意力大图	8448	24404	184	28	7
矩阵乘小图	34944	0	46	227	7
矩阵乘大图	460800	0	255	3345	15

两个卷积图和两个矩阵乘图的额外搬运均来自一级缓存。卷积小图有 28 个不同对象被换出，形成 56 次事件；大图有 690 个不同对象，形成 2116 次事件，单个对象最多换出 9 次。大图的重复事件数为 1426，表明搬运并非完全由大量互不相同的对象各换出一次构成。共享对象在不同需求之间被移出再恢复，其未来使用跨度与分支执行顺序因而值得共同考虑。

矩阵乘小图仅有 46 个不同对象被换出，却形成 273 次事件；大图相应为 255 个对象、3600 次事件，最大重复次数为 15。结合表 7 中一级缓存主导的逻辑峰值，可以把问题一的长时间保留需求与问题二的反复恢复联系起来：有限容量方案通过多次恢复服务同一批持续存在的逻辑对象。该统计定位了当前方案的开销分布，进一步降低搬运量需要重新安排对象的多次使用或选择其他驻留关系。

注意力小图的 23 次换出涉及 23 个不同对象，全部发生在统一缓存，总搬运量为 3620。注意力大图则有 179 次统一缓存换出及 33 次一级缓存换出，对应搬运量 24404 和 8448。这一差别与峰值组成相呼应：小图总峰值位置的一级缓存为 3072，低于 4096，而统一缓存为 1044，已超过 1024；大图的相应值为 5120 与 2084，两种类型都出现较强需求。分类型统计因而比单个总峰值更直接地说明换出发生的位置。

6.7 真实驻留事件与连续地址复用

图 2 按扩展调度序列重放注意力小图的物理驻留。横轴包含新增换出换入节点，纵轴分别是一级缓存和统一缓存的占用量；虚线表示相应物理容量。每次申请或换入使占用上升，每次释放或换出使占用下降。此图用于观察逻辑序列中的空间状态，执行周期上的等待将在下一节通过流水图分析。

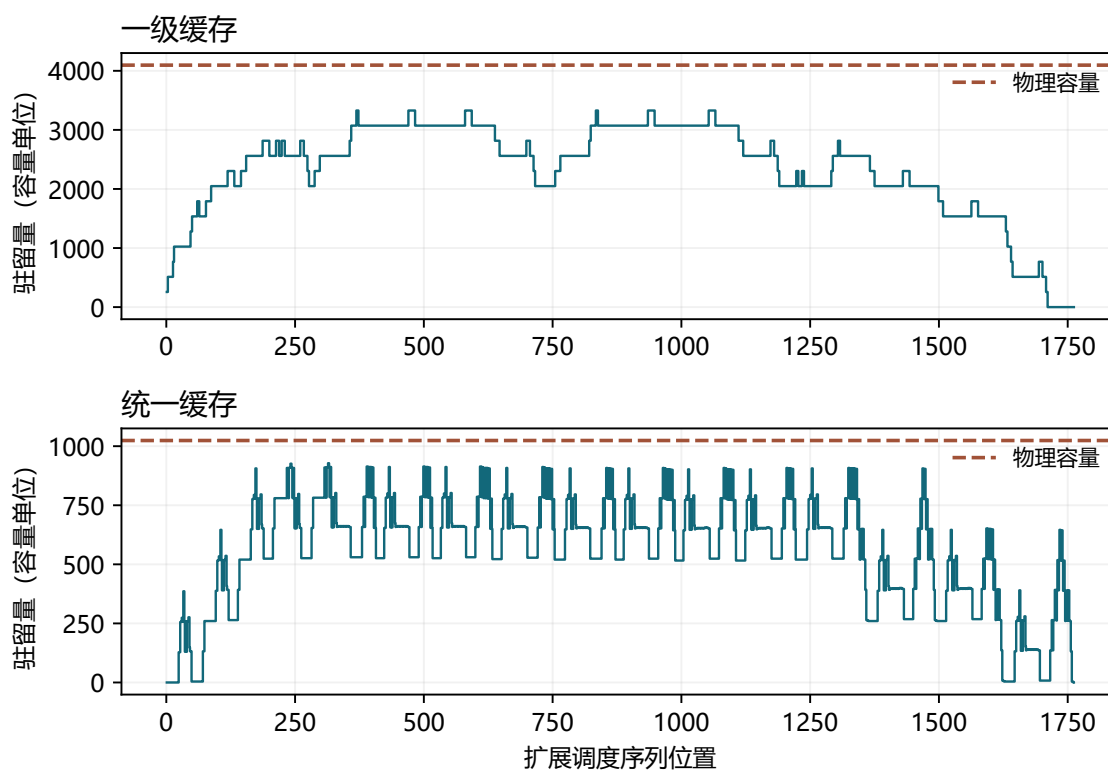


图 2: 注意力小图问题二方案的分类型物理驻留

第一处换出发生在扩展序列第 237 个位置。此前统一缓存已占 908，空闲总量为 $1024 - 908 = 116$ ，下一个对象 87 需要 128，因此该请求首先遇到总量不足。算法将大小 128 的对象 78 从 $[520, 648)$ 换出，占用降至 780，再在同一区间申请对象 87，占用回到 908。节点 1716 对应这次换出，节点 171 对应对象 87 的原始申请。具体位置与地址见表 10。

表 10: 注意力小图首个换出附近的统一缓存事件

位置	节点	操作	对象	大小	偏移	事后占用
234	170	申请	86	128	648	908
237	1716	换出	78	128	520	780
238	171	申请	87	128	520	908
240	174	申请	88	2	908	910
241	175	申请	89	16	910	926
243	1233	释放	89	16	910	910
244	1718	换出	88	2	908	908
245	1720	换出	77	128	780	780
246	177	申请	90	128	780	908

这一局部包含大小为 2、16、128 的不同对象。小对象释放可以增加空闲总量，但未必立即提供适合大对象的连续段；分配过程因此在每次状态变化后重新扫描地址。在第 244、245 个位置连续换出两个对象后，大小为 128 的对象 90 被放入 [780, 908)。这些事件显示，搬运次数与搬运量承担不同的解释功能：换出小对象增加事件数，却可能只贡献很小的数据量。

对象 78 的逻辑生命在换出后仍继续，其数据在配对换入完成后才能再次被使用。与此同时，原地址已经交给对象 87。下一问据此加入“对象 78 本段换出结束先于对象 87 申请”的边，把本节序列中的合法复用转化为时间上的等待约束。地址方案由此既决定空间可行性，也参与流水并行关系的形成。

7. 搬运量约束下的流水优化

7.1 联合目标与时间递推

在有限缓存下，串行输出一个合法节点序列不等于硬件串行执行全部操作。同一执行单元的指令互斥，不同单元则可以并行；能否重叠取决于数据和地址依赖。以问题二的搬运量 D_2 为预算，问题三采用

$$\min T, \quad D \leq D_2, \quad T \geq F_v (v \in V_R), \quad (15)$$

并保留问题二的全部拓扑、驻留和地址约束。本文将题面“不显著增加”具体化为零增量预算，以问题二选定方案的搬运量作为上限，在这一条件下优化完成时间。

构造驻留依赖图 $G_R = (V_R, E_R)$ 。除原始边外，段开始节点指向该段使用者，全部使用者指向段结束节点，换出指向配对换入。逐地址单元跟踪上次结束驻留的节点；再次占用时，添加从旧段结束到新段开始的边。若连续多个段使用同一地址，只需连接相邻段，前后关系可由路径传递。该做法也覆盖部分区间相交，不能仅比较起始偏移是否相等。

给定 G_R 的一个拓扑序，记同一单元上序列中紧邻 v 的前一操作为 $h(v)$ ，则

$$S_v = \max \left\{ 0, \max_{u:(u,v) \in E_R} F_u, F_{h(v)} \right\}, \quad (16)$$

$$F_v = S_v + c_v, \quad T = \max_{v \in V_R} F_v. \quad (17)$$

无相应前项时取零；管理节点不加入执行单元互斥队列。式 (16) 取所有必要等待的最大值，所以同时满足非负、依赖和单元互斥。按拓扑序归纳，任何保持相同各单元指令顺序的可行时间表，都不能把当前节点提前于该最大值。因此，在各执行单元指令顺序固定时，该递推给出满足依赖约束的最早时间表。

将结构依赖就绪时刻记为 $A_v = \max_{u:(u,v) \in E_R} F_u$ ，单元可用时刻记为 $P_v = F_{h(v)}$ 。二者分别表达数据及地址准备情况和单元队列状态。若 $A_v > P_v$ ，该单元在前一操作结束后等待结构依赖；若 $P_v > A_v$ ，节点已经具备结构条件，但要等待同单元前项完成。相等时两个条件同时达到边界。由于前驱自身也会受其他资源约束，局部两类时刻用于定位直接约束，完整因果关系仍沿依赖路径传递。

本节优化的对象是各单元指令的相对次序。原始总序列中的许多相邻节点分属不同单元，只要不存在数据或地址依赖，就可在时间轴上重叠。将总序列简单相加会把这些可并行区间全部串行化；式 (16) 则为每个单元保留独立时钟，同时通过前驱完成时间连接各单元。

7.2 固定驻留图重排为何保持可行

只保留 E_R 中的结构依赖，暂不固定同一执行单元的相邻次序，就获得一个可改变流水顺序的搜索空间。对每个地址单元，原方案的驻留段由“旧段结束先于新段开始”连接成链；每个段的使用者又位于开启与关闭之间。任意新的 E_R 拓扑序均保留这两种关系，所以既不会把使用者移动到段外，也不会让同一地址的两个段交叠。不同地址允许交换次序，不影响空间互斥。随后按式 (16) 加入新的单元顺序，时间上仍然可行。

这里固定的是对象、偏移及各地址上的驻留先后关系，而不是原序列中所有节点的总顺序。换出事件的对象和次数不变，式 (12) 的每一项随之不变，故固定驻留图重排严格保持 D 。这一性质比事后观察搬运量相同更强，但只适用于固定事件集合的阶段；地址重分配本身仍须单独接受预算检查。

7.3 驻留依赖的局部构造与时间小例

以问题二中对象 78 到对象 87 的地址复用为例。对象 78 在偏移 520 驻留，换出节点 1716 关闭其当前驻留段；对象 87 的申请节点 171 随后占用相同区间。因此构造边 $1716 \rightarrow 171$ 。对象 78 在该段中的全部使用者指向 1716，171 再指向对象 87 在新段中的使用者，形成“旧使用—换出一新申请—新使用”的顺序链。

若两个区间仅部分重叠，也按每个重叠地址单元连接前一结束节点与后一开始节点。例如旧区间为 $[4, 10)$ ，新请求为 $[8, 12)$ ，地址 8、9 上的前一释放都必须完成。程序把相同的节点对合并成一条边，重复检查不同单元不会重复增加等待。对于完全不相交的地址，则无需加入这一类边，保留了跨单元重叠的机会。

换入节点还必须等待对应换出。对于一个对象的多次换出，每段的使用者均在该段开始之后、结束之前，相邻段由换出到换入的配对关系连接。由这些路径可以推出早期使用先于后续换出、早期换入先于后续使用，逐段构造因而覆盖整个逻辑生命。原始依赖边同时保留，物理地址约束是在原图基础上增加的限制。

再考虑两个操作的手算例。输入搬运单元执行 u ，耗时 10；输出搬运单元执行 v ，耗时 20，原图中二者无依赖。若两对象使用不同地址，则 $S_u = S_v = 0$ ，完成时间为 20。若它们先后复用同一区间，旧段释放到新段申请的边使 v 等待 u ，此时 $S_v = 10$ ，完成时间为 30。这两个结果已经通过独立重放检验，差别全部来自地址关系。

该例还说明零时长管理节点的作用。申请与释放本身不增加工作量，却能把一段计算的完成约束传给下一段。删除零时长节点时必须保留其传递关系；否则可能错误地得到两个占据相同地址的操作并行执行。本文保留管理节点参加最长路递推，使地址安全与时间计算使用同一组结构边。

7.4 候选扩展与列表调度

首先扩展原始候选：增加两种按估计最早开始时间与新增容量加权的操作排序，窗口均为 64，新增容量权重分别为 0、1 周期/容量单位；对八种顺序的距离换出方案分别尝试地址轮换。轮换从上次分配末端继续查找空闲区间，以改变地址复用形成的依赖。它同时可能改变碎片和换出次数，因而不能声称轮换天然保持搬运不变。初始候选共二十四个，仅保留满足式 (15) 者，从中取得时间最小方案。

其次分别以问题二方案及扩展候选的最佳时间方案为起点，构造固定驻留图。按逆拓扑序计算

$$r_v = c_v + \max_{u:(v,u) \in E_R} r_u, \quad (18)$$

无后继时最大项取零。每个执行单元维护以 r_v 降序排列的就绪队列，管理节点就绪后立即处理。比较两种选择：其一在各队首中选择估计可开始时间最早者，同分优先较大的 r_v ；其二直接优先较大的 r_v 。新顺序必须再次通过完整依赖与地址重放，只有真实 T 下降才替换原方案。

调度优先级借鉴 HEFT 的向上秩思想 [4]。本题将执行单元作为节点的给定属性，以固定节点时长和驻留依赖计算式 (18)。较大的 r_v 表示从当前节点出发还有较长的依赖工作链，优先安排它有助于避免长链持续推迟。本文在既定执行单元上采用追加式列表调度，性能由本题候选的完整重放评价。

设 $N' = N + 2M$ 。路径计算与边遍历为 $O(N' + |E_R|)$ ，优先队列维护为 $O(N' \log N')$ ；固定数量单元的队首比较为 $O(N')$ 。若计入逐单元地址依赖构造，还需 $O(N'C)$ 的保守扫描开销。该策略没有回溯改变已保留的地址链，所以算法高效但搜索受限。

7.5 列表调度的执行步骤与候选接受

构造驻留图后，首先计算各节点入度与下游秩，并把入度为零的节点加入其所属单元的优先队列。管理节点使用单独的就绪队列；它们一旦满足结构前驱，就按已有前驱完成

时间确定自身时刻，随即更新后继入度。普通操作在各单元队列内按下游秩选择队首，因此每轮跨单元比较的候选数受硬件单元数量限制。

对各队首 v ，用已经确定的前驱完成时间和当前单元时钟计算 $\hat{S}_v$ 。最早启动规则选择 $\hat{S}_v$ 最小者，并在相同启动时刻下优先较大 r_v ；路径优先规则则直接以较大 r_v 为主要依据。输出一个操作后，只更新它所在的单元时钟，同时将后继剩余入度减一。所有节点被输出即终止，无就绪节点但仍有未输出节点时对应依赖构造异常，候选应被拒绝。

两种规则面对的是不同局部取舍。若一个单元当前可以启动短链操作，而另一个单元的长链操作需要等待较晚的数据，最早启动规则先处理前者，路径优先规则可能先确定后者。确定顺序本身不会使不同单元共享同一时钟，但会影响以后在同一单元上排在前面的操作。这种影响通过后续队列状态继续传播，最终优劣由全部节点完成时间决定。

节点进入就绪队列表示其结构前驱已经被安排，并不表示前驱在物理时间零点已经结束。其预计开始时刻始终取这些前驱的实际完成时间，避免将“拓扑可选”和“此刻可执行”混淆。下游秩在固定驻留图上预先计算，队列状态随已经安排的节点变化；两者分别提供长期链条优先级和当前可行性约束。

表 11 给出候选的两层筛选。第一层允许改变顺序与地址，搬运量需显式比较；第二层固定事件和地址链，在结构上保持搬运量。每次新时间表仍以独立重放检查地址使用、原始依赖和单元互斥，只有满足预算且时间更短时才更新当前方案。

表 11: 问题三候选生成与接受条件

阶段	可改变的决策	接受条件
扩展候选	操作顺序、地址起点及牺牲事件	空间时间可行， $D \leq D_2$
固定驻留重排	既定结构边下各单元相对顺序	保留事件和地址链，真实 T 下降
最终择优	从原方案及可行改进中选择	取预算内最短完成时间

以问题二方案为起点的支路能够直接检验固定驻留重排的作用；以扩展候选最佳方案为起点的支路则允许先改变地址关系，再压缩其流水。两条支路共用同一搬运预算，原问题二方案始终保留，因此最终时间不会因尝试更复杂的策略而被迫恶化。具体节省来自哪一支路，由实际分阶段结果判定。

7.6 分阶段结果与改善来源

表 12 以问题二方案为共同基准，分别考察候选扩展与固定驻留图重排。 T_A 为扩展初始候选后的最佳时间， T_F 为从问题二方案出发固定驻留图重排的最佳时间； T_3 在两种起点、两种排序规则与已有方案间选优。六图最终 $D_3 = D_2$ ，搬运量对应表 8。

表 12: 预算不增加条件下的时间对照（周期）

计算图	T_2	T_A	T_F	T_3	降幅/%
卷积小图	472038	472038	465892	465892	1.30
卷积大图	1210619	1210619	1044075	1044075	13.76
注意力小图	47509	47509	37744	37744	20.55
注意力大图	225902	225902	150499	150499	33.38
矩阵乘小图	194331	177625	186969	173274	10.84
矩阵乘大图	1800001	1765989	1758881	1749773	2.79

卷积与注意力四图的 $T_A = T_2$ ，而 $T_F = T_3 < T_2$ ，其时间改善出现在固定驻留图重排支路。矩阵乘小图则先由扩展候选从 194331 降至 177625，再降至 173274；从问题二方案直接重排达到 186969。后者节省 7362，扩展后再重排共节省 21057，两条支路的最终时间相差 13695 周期。矩阵乘大图也呈现扩展后再重排更优的次序，说明在这两个实例中，起点所保留的地址关系对后续列表调度具有实际影响。

为比较各算例的相对优化收益，图 3 统一以各图自己的 T_2 为分母，纵轴为 $100(T_2 - T)/T_2$ 。相同百分比表示相对于各自基准节省的时间比例，绝对周期数由表 12 给出。

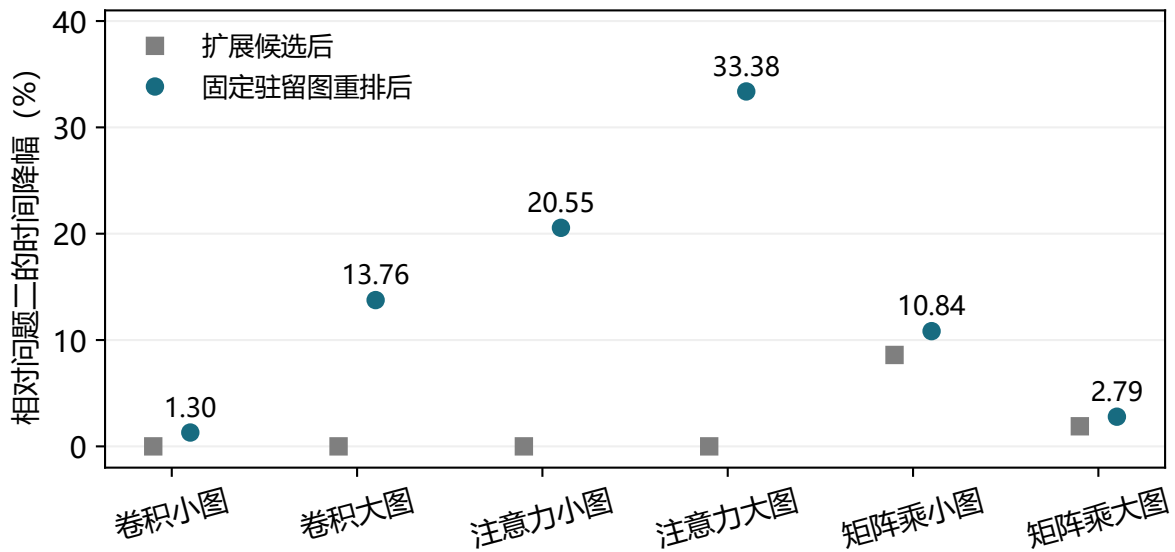


图 3: 扩展候选与最终重排的时间改善

注意力大图降幅 33.38% 最大，矩阵乘大图降幅为 2.79%。前者节省 75403 周期，后者节省 50228 周期，相对收益排序与绝对节省量承担不同含义。为进一步判断各图当前方案的余量，下面将完成时间与工作量及驻留路径下界比较。

7.7 分层下界及瓶颈解释

令 $P(G)$ 为原始图按节点时长计算的最长路径, W_p^0 为单元 p 的原始工作量。忽略全部新增搬运和地址限制, 得到对联合问题也成立的弱下界

$$L_0 = \max\{P(G), \max_p W_p^0\}. \quad (19)$$

固定换出事件后, 记包括新增操作的工作量为 W_p , 则 $L_{\text{work}} = \max\{P(G), \max_p W_p\}$ 是该事件集合下的下界。进一步固定驻留图, 令 $P(G_R)$ 不包含候选单元排序边的最长路径, 得到

$$L_R = \max\{P(G_R), \max_p W_p\}, \quad g_R = 100 (T_3/L_R - 1) \%. \quad (20)$$

证明只需注意两点: 任一依赖路径必须串行完成, 同一单元的全部工作也不能相互重叠。取二者最大值不超过任何合法总时间。加入已选单元排序边虽会得到更长路径, 却把搜索决策固定成现有答案, 不能再用来证明其他单元顺序也必须如此缓慢。

表 13 列出三个层次。 g_R 评价保持当前驻留图时的时间上界与下界之差; 更换换出集合或地址关系, 对应的 L_{work} 和 L_R 随之重新计算。

表 13: 最终方案的分层时间下界及固定驻留差距

计算图	L_0	L_{work}	L_R	T_3	$g_R/\%$
卷积小图	348677	348677	458852	465892	1.53
卷积大图	409566	761650	835577	1044075	24.95
注意力小图	25600	25600	31953	37744	18.12
注意力大图	104960	104960	127789	150499	17.77
矩阵乘小图	65536	152438	152438	173274	13.67
矩阵乘大图	524288	1628000	1628000	1749773	7.48

卷积小图的原始工作下界为 348677, 固定驻留路径使下界提高到 458852, 最终 465892 高出 7040 周期。在当前驻留关系下, 进一步重排的最大可能收益受这 7040 周期限制, 因此改变地址链是继续扩大搜索范围的一种方向。卷积大图相对固定驻留下界仍有 24.95% 的差距, 可在保留驻留图的条件下继续研究更强的单元排序; 下界与实际可达时间之间的距离还包含多资源约束的影响。

表 14 进一步用最大工作量单元的忙碌比例解释不同图的表现。 W_p 在问题二和最终方案中相同, 忙碌比例定义为 W_p/T , 其余完成窗口合并计为等待。该指标先定位单元的总时空隙, 再由具体节点的就绪时刻分析局部直接约束。

表 14: 最大工作量单元的忙碌比例

计算图	最大工作量单元	W_p	$W_p/T_2/\%$	$W_p/T_3/\%$
卷积小图	CUBE	348677	73.87	74.84
卷积大图	MTE2	761650	62.91	72.95
注意力小图	VECTOR	25600	53.88	67.83
注意力大图	VECTOR	104960	46.46	69.74
矩阵乘小图	MTE2	152438	78.44	87.98
矩阵乘大图	MTE2	1628000	90.44	93.04

注意力大图的向量单元从 46.46% 提升至 69.74%，工作量仍为 104960 周期，改善来自总时间窗口的收缩。矩阵乘大图的输入搬运单元由 90.44% 升至 93.04%，其 1628000 周期的工作量形成较强限制。在保持这份搬运事件集合时，总时间至少覆盖这些工作；进一步降低工作量则需要回到换出与恢复方案的选择。

为观察等待分布，图 4 展示注意力小图在相同横轴上的两份完整流水。细条表示实际操作区间，空白表示该单元未执行操作；换出为零周期时不画成有宽度的条块，但其依赖仍参与计算。

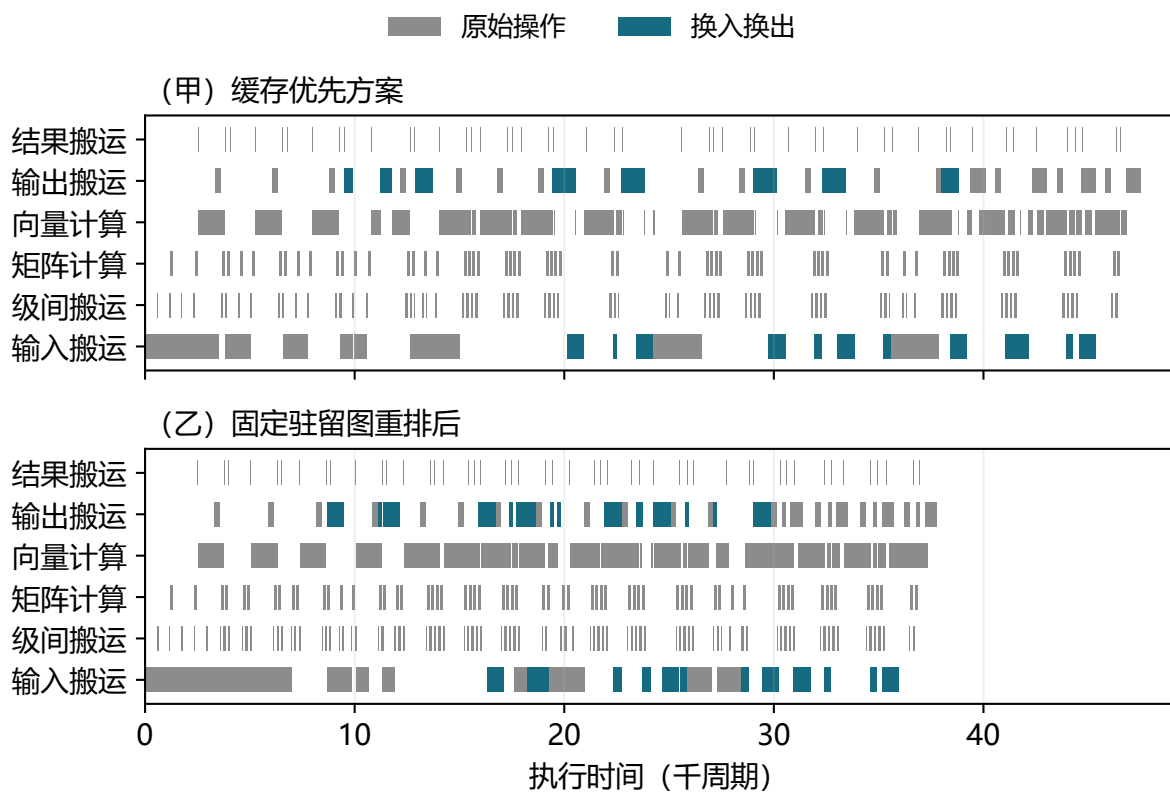


图 4: 注意力小图的优化前后流水 (同一时间尺度)

两方案搬运量同为 3620,总时间从 47509 降至 37744 周期,向量单元忙碌比例从 53.88% 升至 67.83%。完整流水反映多个阶段的重叠与最终完成窗口,局部依赖则在下一小节的放大图中展开。

7.8 注意力局部窗口：依赖就绪如何传递

选择 8500 至 11500 周期的共同时间窗,图 5 放大两份方案。横轴保持相同尺度,原始操作与新增搬运分色表示,并标注向量单元相关节点 136、173 及节点 173 的紧前依赖 172。该窗口来自原时间表截取,所有区间保留实际开始与结束时刻。

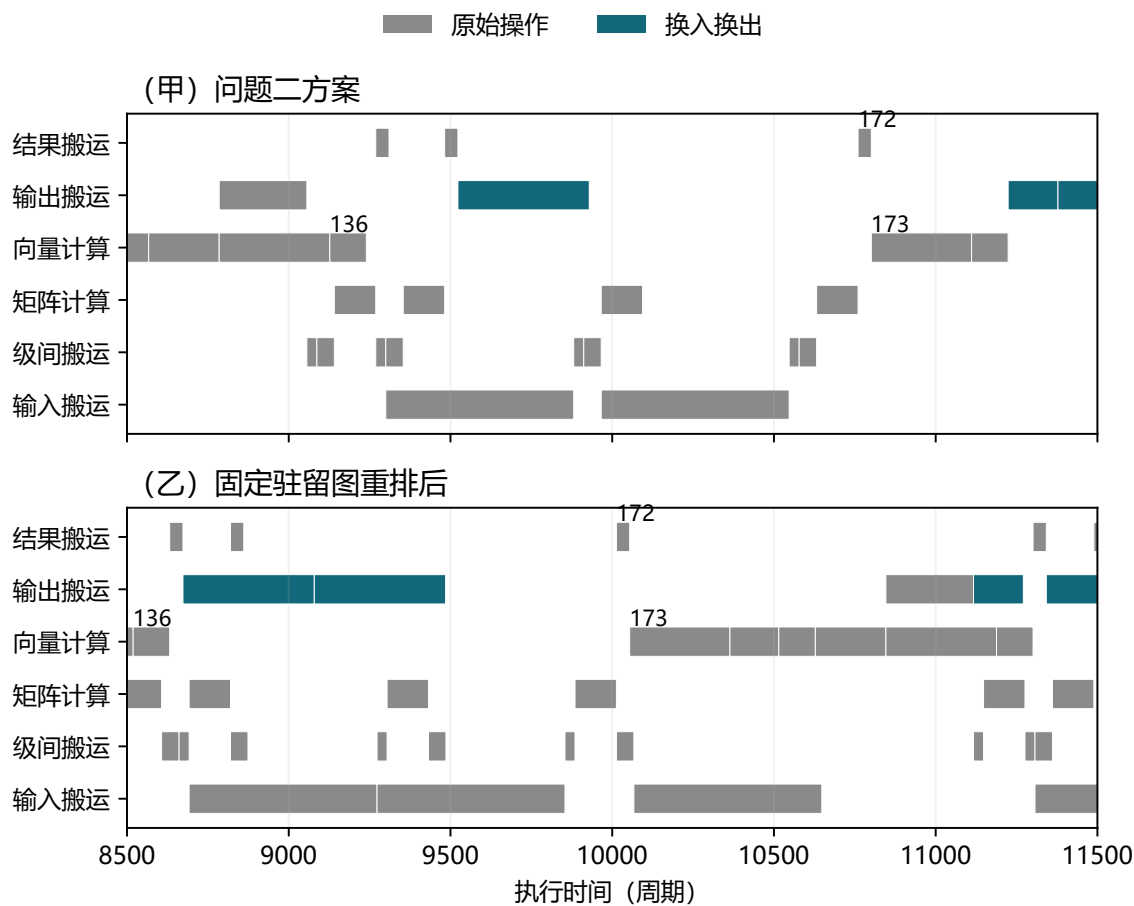


图 5: 注意力小图同尺度局部流水与节点标识

问题二方案中,向量单元前一节点 136 在 9240 周期结束,行最大值节点 173 等待到 10801 周期才开始,其耗时为 310。按时间递推,该节点的结构依赖就绪时刻为 10801,达到这一时刻的前驱为节点 172。因此节点 136 与 173 之间的空隙为 $10801 - 9240 = 1561$ 周期,直接限制来自结构依赖。

重排后,节点 136 在 8632 周期结束,节点 172 所决定的结构就绪时刻提前到 10054,节点 173 随即执行至 10364。该处空隙为 $10054 - 8632 = 1422$ 周期,比原方案减少 139。与此同时,节点 173 本身开始时刻提前 747 周期,这一提前量包括前序过程整体前移和局部空隙缩短两部分。由此,局部等待变化与累计完成提前量能够分别解释。

表 15: 向量节点的直接等待条件（周期）

节点	方案	前项完成	依赖就绪	开始	空隙
75	问题二	3794	5270	5270	1476
75	问题三	3771	5061	5061	1290
124	问题二	6517	7993	7993	1476
124	问题三	6308	7385	7385	1077
173	问题二	9240	10801	10801	1561
173	问题三	8632	10054	10054	1422
222	问题二	12608	14053	14053	1445
222	问题三	11301	12355	12355	1054

表 15 还列出节点 75、124、222 的同类对照。四个节点在两方案中均由结构依赖就绪时刻决定开始，直接空隙分别缩短 186、399、139、391 周期。多个局部空隙的缩短与整体向量忙碌比例提高一致，而节点的提前量又会通过后继传播到更远位置。这些局部观察用于说明具体执行过程，各项空隙相互受前驱链影响，总完成时间仍按全图递推计算。

首次向量操作节点 22 的开始时刻仅从 2547 提前到 2524，变化为 23 周期；后续节点 222 的开始时刻则从 14053 提前到 12355，变化为 1698。这表明整个 9765 周期的最终节省包含后续流水组织的持续改善，而非只将启动阶段整体平移。固定驻留重排保持全部换出对象和事件次数，因而这些时间变化发生在相同搬运量条件下。

综上，问题三利用地址链允许的可交换次序改善各单元流水。卷积与注意力图主要受益于固定驻留重排，矩阵乘图进一步受益于扩展地址候选。分层下界给出各固定问题的解质量范围，整体忙碌比例和局部就绪对照则说明完成窗口如何缩短，形成从约束、算法到实际时间表的对应关系。

8. 敏感性、边界与验证

8.1 窗口参数的可辨识性

候选规则全部一致应用于六个图，不按算子名称选用专门参数。为分离窗口影响，表 16 只比较释放权重 $\alpha = 1$ 、距离换出、最佳适配都固定时的 $W = 32$ 和 $W = 128$ 。 H_W 是原始序列峰值， D_W 是随后分配的额外搬运量。

表 16: 固定其他规则时的窗口对照

计算图	H_{32}	H_{128}	D_{32}	D_{128}	顺序相同
卷积小图	21762	21762	95564	95564	否
卷积大图	157630	180030	407112	439760	否
注意力小图	5536	5536	24896	24896	是
注意力大图	19008	19008	129512	129512	否
矩阵乘小图	14208	24704	85632	152832	否
矩阵乘大图	39680	52352	611968	719488	否

矩阵乘小图扩窗后，峰值从 14208 升至 24704，搬运量从 85632 升至 152832，说明扩大就绪选择范围不会单调改善结果。窗口改变的是每一步的可选集合，不是对已完成全局方案作包含关系扩展；一旦局部选择改变，后续状态也改变，所以“更多选择”不保证最终更优。

卷积小图与注意力大图两种窗口的峰值和搬运量相同，但完整序列并不相同，时间分别为 642017 与 640953、485807 与 486988 周期。这是目标统计量无法区分某些顺序差异的例子，不能据此认定参数未生效。注意力小图的两份完整序列确实相同，只能认定这两个窗口值在该实例上落在相同输出区间；本轮未记录每次候选竞争，尚不能区分窗口未约束就绪集合与最优局部选择恰好一致这两种原因。

此外，(64, 2) 同时改变窗口和权重，因此只作为另一个算法候选，不把其与 (32, 1) 的差异单独归因于释放权重。本轮未进行容量扰动实验：改变容量意味着改变题目边界条件，现有结果不足以支持对任意缓存规模的稳健性结论。

8.2 手算对照与错误注入

可行性检查从导出的序列、偏移与换出列表重新读入，不调用候选生成或分配函数。它检查原始及新增节点恰好出现一次、原始边顺序、换出配对、每次地址申请的整数边界与逐单元占用、操作使用时的驻留状态，最后重新计算流水和搬运量。问题三与问题二的十二份完整方案及六份问题一序列均完成相应重算。

为了防止执行时间计算把所有操作错误串行化，构造两个无依赖操作，分别在输入、输出搬运单元执行 10 和 20 周期，缓冲区大小均为 4。地址互不相交时，正确总时间为 $\max(10, 20) = 20$ ；复用同一地址时，前段释放到后段申请形成依赖，正确总时间为 $10 + 20 = 30$ 。这两个答案只差地址关系，可同时检验并行能力和地址等待是否生效。

对大小为 4 的输入复制缓冲区，一次换出搬运量为 4，换入耗时为 $2 \times 4 + 150 = 158$ ；按测试给定的其他操作和依赖，完整总时间为 188。对应非输入复制对象，搬运量为 8，两次搬运各耗时 158，完整时间为 346。这些手算测试覆盖零耗时换出和双向计费。再分别注入地址重叠、拓扑逆序、越界偏移及节点重复，检查程序均拒绝对应附件。拒绝非法输入只是验证的一部分，不替代对合法实例的数学建模判断。

8.3 解释边界与尚未解决的问题

本题附录 C 的地址复用描述直接针对 ALLOC/FREE；本文将其延伸为驻留段结束到开始的依赖。若把已换出的缓冲区仍视为占用地址直到最终 FREE，就无法表达换出释放空间的目的，并可能生成环。因此采用驻留段解释有物理上的一致性，但尚无官方评价器作交叉确认。相关时间结论均以此解释为前提。

对搬运量，本文只有非负这一平凡下界，没有得到六图的最小必要换出集合；对驻留峰值，式 (5) 也较弱。这使模型能证明方案满足约束，却不能证明搬运和峰值接近全局最优。时间下界更有辨识力，但强下界仅针对固定事件或驻留图。三种解质量保证的范围不同，不应合并成一个“算法最优”结论。

六图都是题面示例，没有独立保留的泛化测试集；相同规则跨图应用，只证明实现没有写入算子专用分支。进一步的通用性评价需要不同规模和不同共享数据结构的新图。求解与重放虽分开实现，仍共享输入解析和题意解释，不能排除共同理解偏差。真实芯片测量也未进行，周期改善不等同于实测吞吐加速。

9. 模型评价与结论

本文围绕三个递进目标，分别把节点顺序、物理空间和流水等待显式化。生命周期收缩在固定操作顺序下不增峰值；连续分配通过完整驻留状态与有代价的恢复机制获得可行方案；固定驻留图重排保持地址先后和搬运事件不变，从而将时间优化转化为受资源约束的拓扑排序。

六图中，问题一只有卷积小图相对收缩后的编号基准降低峰值，降幅为 7.62%。问题二的搬运量依次为 44766、75546、3620、32852、34944 和 460800。问题三在这些搬运量均不增加的条件下，将总时间降低 1.30% 至 33.38%；其中注意力大图从 225902 降至 150499 周期。阶段比较表明，四图的改善来自固定驻留重排，两个矩阵乘图还需要先改变地址分配候选。

在当前六图与驻留段模型下，缓存方案确定后仍存在可利用的流水优化空间。卷积小图距固定驻留下界为 7040 周期，进一步研究可以转向地址关系；卷积大图与该下界相差 24.95%，适合继续扩展单元顺序搜索。矩阵乘大图的输入搬运工作量为 1628000 周期，可优先研究重复换出对象的使用组织。不同方向分别对应地址链、资源排序与搬运事件三个层次。

过程分析进一步给出这些方向的具体依据。卷积小图的选定峰值由一级缓存 6912 与零级右输入缓存 72 组成，局部缓存贪心却在零级右输入缓存积累了 16384 的逻辑需求；矩阵乘大图的 3600 次换出涉及 255 个不同对象，反复恢复是当前方案的重要开销；注意力小图在固定搬运量条件下，多处向量节点的依赖等待缩短并向后传播，最终完成窗口减少 9765 周期。这些结果把汇总指标落实到实际对象、事件及时间窗口。

方法的主要价值是将三个目标组织为可逐层检验的完整方案：生命周期收缩提供统一基准，连续分配保证物理空间与恢复状态合法，固定驻留调度利用地址链保留的并行机会。全局峰值和搬运量的紧下界、不同图结构上的泛化表现以及真实芯片对应关系，构成后续研究的三个重点。现阶段结论适用于题面六图及本文明确的驻留段复用解释。

参考文献

- [1] 中国研究生数学建模竞赛组织委员会. 2025 年中国研究生数学建模竞赛 A 题: 通用神经网络处理器下的核内调度问题及附件. 2025.
- [2] Wilson P R, Johnstone M S, Neely M, Boles D. Dynamic Storage Allocation: A Survey and Critical Review. International Workshop on Memory Management, LNCS 986, 1995: 1–116. DOI: 10.1007/3-540-60368-9_19.
- [3] Bélády L A. A study of replacement algorithms for a virtual-storage computer. IBM Systems Journal, 1966, 5(2): 78–101. DOI: 10.1147/sj.52.0078.
- [4] Topcuoglu H, Hariri S, Wu M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing. IEEE Transactions on Parallel and Distributed Systems, 2002, 13(3): 260–274. DOI: 10.1109/71.993206.
- [5] 中国研究生数学建模竞赛组织委员会. “华为杯”第二十二届中国研究生数学建模竞赛人工智能工具及输出使用规定. 2025. <https://cpipc.acge.org.cn/sysFile/downloadFile.do?fileId=e3edba52b87c4bf7922f737c35c14b74>. 访问日期: 2026-09-21.

A. 复算与人工智能使用说明

支撑材料给出三问的完整调度文本。问题一只含原始节点顺序；问题二、三分别提供扩展节点顺序、初始地址偏移与按换出顺序列出的新偏移。新增节点每对由换出及换入组成，重新排列后也保持列表与节点编号一致。复算应从原图和这些文本重新建立依赖、空间状态及执行时间，不能只读取汇总数值。

程序以 Python 实现。图读取、顺序生成、地址分配、流水重排与附件重放分模块组织；候选规则及平局处理均为确定性。复算材料保留全部候选与失败记录，论文表中的最终值来自同一组六图方案。新增过程分析从这些方案重放得到，并以九节点构造例的全部 126 个拓扑序检验生命周期峰值。

研究过程中使用 OpenAI Codex 辅助模型推导、程序实现、结果分析及论文编辑。历史求解回合的精确模型标识和版本发布日期未核实，参赛队员独立复核尚未完成。若用于参赛，队员需依据适用规定理解并核验相关内容，以自身语言形成最终文本，并补齐真实工具记录与使用声明 [5]。